Client Server and Web development

Chapter 3 : Learn to code with JavaScript

I. Introduction

JavaScript is probably **the** main web programming language. It was invented in 1995 by Brendan Eich, who at the time worked for Netscape, which created the first popular web browser (Firefox's ancestor).

JavaScript should not be confused with Java, another language invented around the same time!

The idea was to create a simple language to make web pages dynamic and interactive, since back then, pages were very simple.

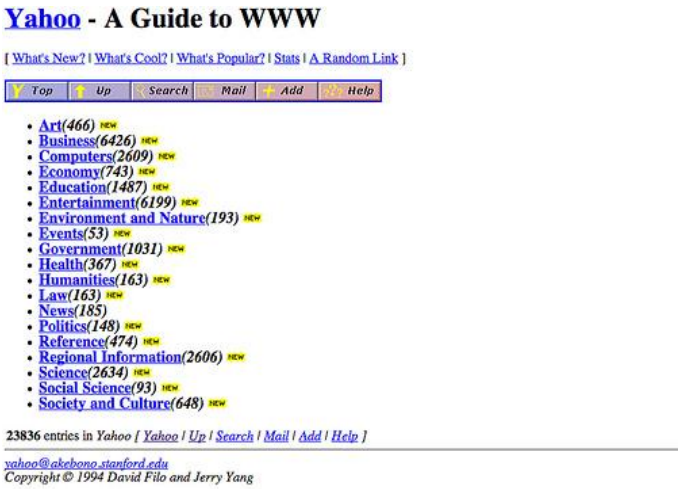


Figure 1 Yahoo's home page circa 1994

Web builders starting gradually enriching their pages by adding JavaScript code. For the code to work, the recipient web browser (the software used to surf the web) had to be able to process JavaScript. This language has been progressively integrated into all browsers, and now all browsers (almost) are able to handle it!

Because of the explosion of the Web and the advent of the **web 2.0** (based on rich, interactive pages), JavaScript has become increasingly popular. Web browser designers have optimized the execution speed of JavaScript, which means it's now a very powerful language.

This led to the 2009 emergence of the **Node.js** platform, which even allows you to create whole server-side web apps in JavaScript. Thanks to a service called MongoDB, JavaScript has even entered the database world (software whose role is to store information).

Finally, the popularity of smartphones and tablets with different **systems** (iOS, Android, Windows Phone) has led to the emergence of so-called cross-platform development tools. They allow you to write a single mobile application that's compatible with these systems. These tools are almost always based on...JavaScript!

In short, **JavaScript is everywhere**. Knowing it will open the doors of the web browser-side programming (known as front-end development), server side development (backend), and mobile development. Not bad!

1.1 JavaScript versioning

JavaScript was standardized in 1997 under the name ECMAScript. Since then, the language has undergone several rounds of improvements to fix some awkwardness and support new features.

This course uses the current version of JavaScript, called ES5, introduced in 2009. The newest version of the language (SS6 / ES2015) will be interesting but is, at the time this course was written, not supported well enough by web browsers

Our website is a bit special because it contains no content that will be displayed by the browser. The purpose of this page is to tell the browser that we want to execute JavaScript file through the<script> tag.

You'll notice a file path (src="../ js / course.js") in each example. We say that the path "points" to this file. The path shown here is the file located in the course.js chapter-3 /js directory. Two dots (..) at the beginning of the path indicate you're going back one level in the directory structure relative to the HTML file itself. When a web browser will attempt to display the course.html page, it will execute the JavaScript code in the file course.js.

1.2 Organise your code

You're almost ready to get started! It's important to set up your basic folder and file structure before actually starting to code. That way, your project will be organized, and you'll be starting off with some good programming habits!

You have to create the project files for this chapter from scratch an example is given in figure 2 bellow :

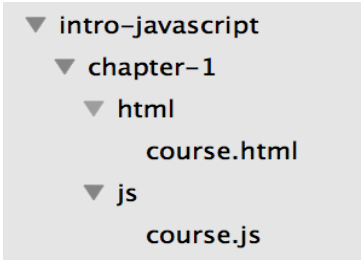


figure 2 : Organizing your code

Each chapter has a folder, which contains an html and a js folder, each of which contains a file. These files are what you can fill out for each chapter, or you can create your examples .

In your first JavaScript file (chapter-3/js/course.js if you're using the file skeleton provided), type the following content:

```
alert("Hello in JavaScript!");
```

and in the the html chapter-3/html/course.html file the following content:

```
<!doctype html>

<html>

<head>

   <meta charset="utf-8">
```

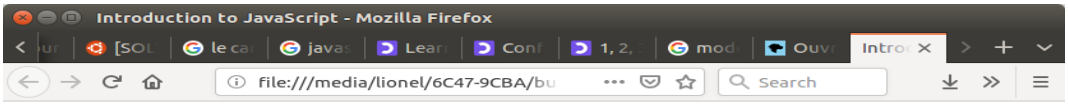
```
    <title>Introduction to JavaScript</title>
</head>
<body>
    <script src="../js/course.js"></script>
</body>
</html>
```

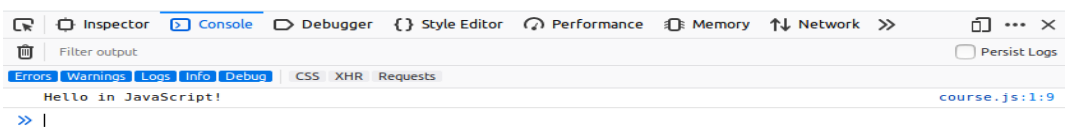
The.js at the end of the name indicates that the file contains JavaScript. All JavaScript files will have this extension.

1.3 Try out your first program

Once you open a web page in a browser, the content will show in the main window, and the associated JavaScript console will show up in the console below (assuming you have it open). For that go to tools/web developer/web console

Before opening your project in a browser, make sure you save both the course.js and course.html files.





II. Fundamentals in javascript

This part will introduce you to the fundamentals of programming including values, types, and program structure.

2.1 Values and types

A value is a piece of information used in a computer program. Values exist in different forms called types. The type of a value determines its role and operations available to it.

Every computer language has its own types and values. Let's look at two main types available in JavaScript.

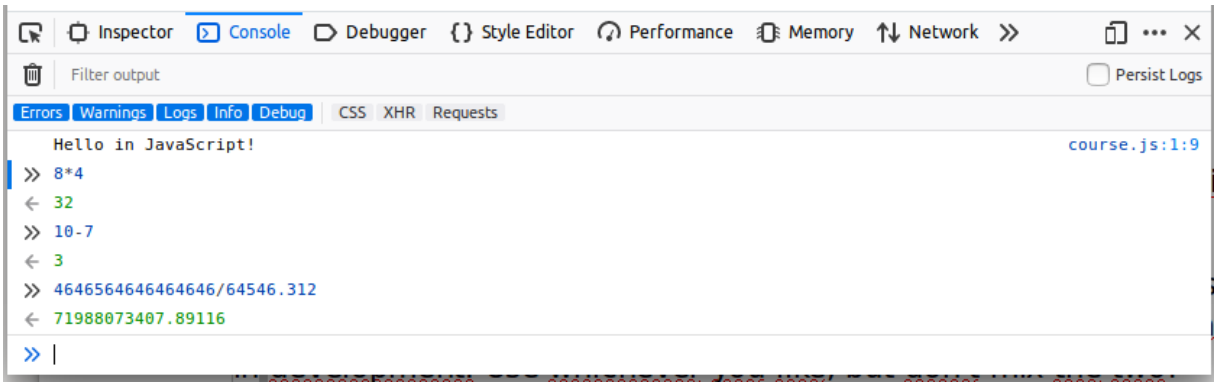
Number

A number is a numerical value (thanks Captain Obvious). Let's go beyond that though! Like mathematics, you can use integer values (i.e. whole numbers) such as 0, 1, 2, 3, etc, or real numbers (i.e. with decimals) for greater accuracy.

Numbers are mainly used for counting. The main operations you'll see are summarized in the following table:

Operator	Job
+	Addition
-	Subtraction
*	Multiplication
/	Division

Most browser consoles allow you to type JavaScript code and execute it. In Firefox simply type 8*4 and press Enter. You get the expected result:32. Test other operations!



Strings

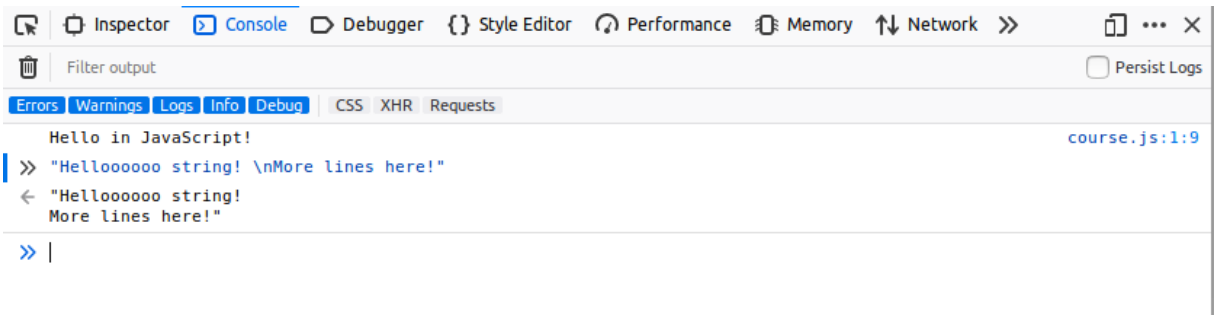
A **string** in JavaScript is a set of characters surrounded by quotation marks, such as "This is a string."

You can also define strings with a pair of single quotes: 'This is a string.' Whether it's best practice to use single or double quotes is a whole political argument in development. Use whichever you like, but don't mix the two!

Always remember to close a string with the same type of quotation marks you started it with.

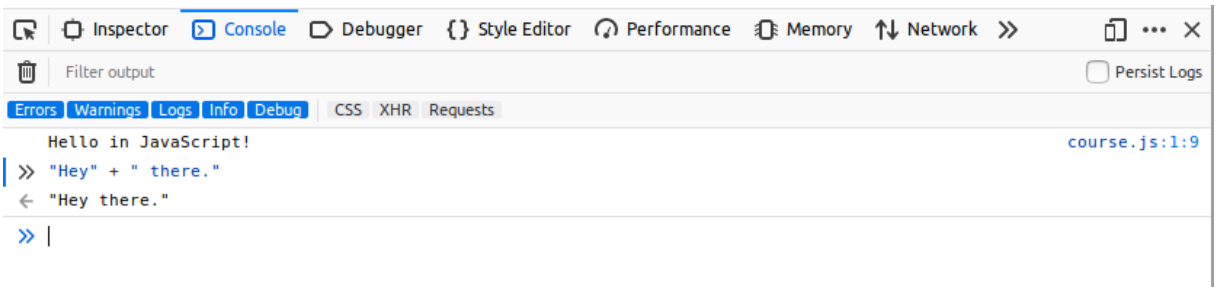
To include special characters in a string, use \ (backslash) before the character you want to add. For example, type\n to add a new line within a string.

In your browser console, enter the line "Hellooooo string! \nMore lines here!" (including the quotes). You'll get a two-line result.



You can not add or remove string values like you'd do with numbers. However, the + operator can be applied to two string values. This will join two strings together, otherwise called **concatenation**.

In the browser console, type "Hey" + " there." You get the result "Hey there."



If you get the result "Heythere." it's because you didn't add a space either after the string "Hey" or before "there."

Displaying values

When you want to display a value from a JavaScript program, use the JavaScript commandconsole.log(). The value you want displayed is enclosed in parentheses and followed by a semicolon. It could, for example, be a number or a string.

You might remember this from the course.js program you created in the previous chapter: it shows "Hello in JavaScript," which is a string.

console.log ("Hello in JavaScript!");

Displaying text on the screen (the famous example "Hello world") is often the first thing you'll do when you learn programming. It's the classic example. You've already taken that first step!

2.2 Program structure

We already defined a computer program as a list of commands telling a computer what to do. These orders are written as text files and make up what's called the "source code" of the program. The lines of text in a source code file are called lines of code.

The source code may include empty lines: these will be ignored when the program executes.

Instructions

Each instruction inside in a program is called a statement. A statement in JavaScript ends with a semicolon, and your program will be made up of a series of these statements.

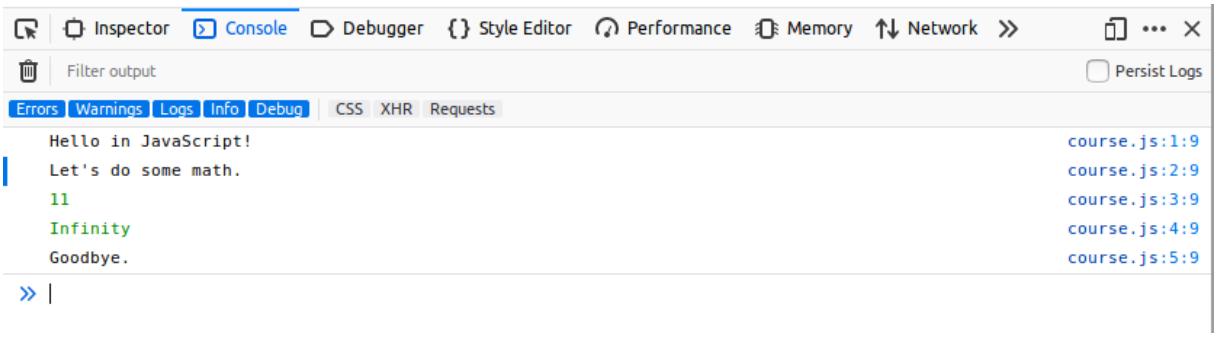
You usually write only one statement per line.

Execution flow

When a program is executed, the instructions in it are "read" one after another. It's the combination of these individual results that produces the final result of the program.

Hop back to Sublime and add the contents below to your course.js file. The instructions in here will be read and executed, line by line.

```
console.log("Hello in JavaScript!");

console.log("Let's do some math.");

console.log(4 + 7);

console.log(12 / 0);

console.log("Goodbye.");
```



Comments

By default, each line of text in the source files of a program is considered a statement that should be executed. You can prevent certain lines from executing by putting a double slash before them: //. This turns the code into a **comment.**

Modify course.js by adding// on several lines. You'll notice the text grays out:

```
console.log("Hello in JavaScript!");
```

```
// console.log("Let's do some math.");

console.log(4 + 7);

// console.log(12 / 0);

console.log("Goodbye.");
```

Re-open your course.html file in firefox, and you'll notice that the commented-out lines no longer appear in the console. As we hoped, they weren't executed!

Comments are great for developers so you can write comments to yourself, explanations about your code, and more, without the computer actually executing any of it.

You can also write comments by typing /* */around the code you want commented out.

```
/* A comment

on multiple

lines */

// A comment on one line
```

2.3 Play with variables

You know how to use JavaScript to display values. However, for a program to be truly useful, it must be able to store data, like information entered by a user. Let's check that out.

Variables

Role of a variable

A computer program stores data using variables. A variable is an information storage area. We can imagine it as a box in which you can put and store things!

Variable properties

A variable has three properties:

- Its name, which identifies it. A variable name may contain upper and lower case letters, numbers (*not* in the first position, ex.1stvariable) and characters like the dollar ($) or underscore (_).
- Its value, which is the data stored in the variable.
- Its type, which determines the role and actions available to the variable.

You don't have to define a variable type explicitly in JavaScript. Its type is deduced from the value stored in the variable and may change while the program runs. That's why we say that JavaScript is a **dynamically typed language**. Other languages, like C or Java, require variable types to be defined. This is called **static typing.**

Declaring a variable

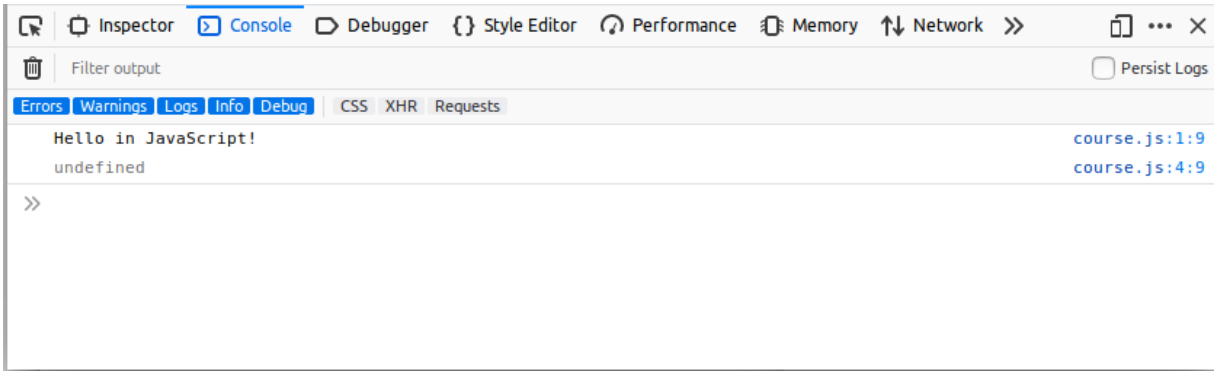
Before you can store information in a variable, you have to create it! This is called **declaring a variable**. Declaring a variable means the computer reserves memory in which it will store the variable. The program can then read or write data in this memory area by manipulating the variable.

Here's a code example that declares a variable and shows its contents:

```
var a;

console.log(a);
```

In JavaScript, you declare a variable with the var keyword followed by the variable name. In this example, the variable created is called a.

Add the above code to your chapter-3 JavaScript file, and open it in Firefox. You'll see the following:



Note that the result is undefined ! This is a JavaScript-type indicating no value. I declared the variable, calling it a but didn't give it a value!

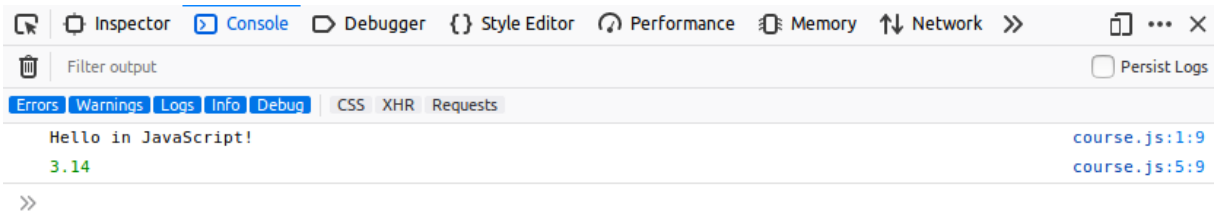
Assign values to variables

While a program is running, the value stored in a variable can change. To give new value to a variable, use the = operator, called an **assignment operator**.

Check out the example below:

```
var a;

a = 3.14;

console.log(a);
```

Here's what you'd get as a result.



We modified the variable by assigning it a value. a = 3.14 means a received the value 3.14.

Be careful not to confuse the assignment operator = with mathematical equality! You'll soon see how to express equality in JavaScript.

You can also combine declaring a variable and assigning it a value in one line. Just know that, within this line 1, you're doing two different things at once:

```
var a = 3.14;

console.log(a);
```

Increment a number variable

You can also increase or decrease a value of a number with += and ++. The latter is called an **increment operator**, as it allows incrementation (increase by 1) of a variable's value.

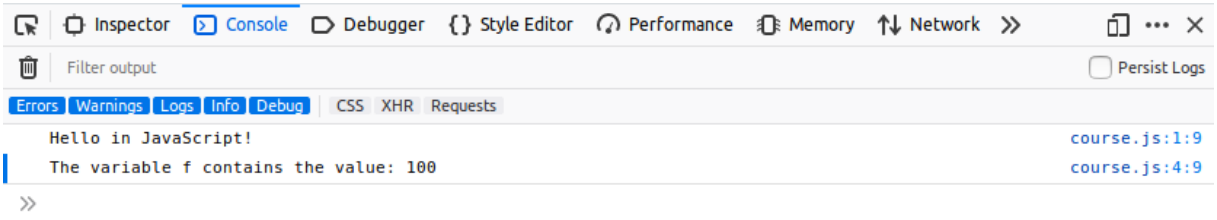
In the following example, lines 2 and 3 each increase the value of variable b by 1.

```
var b = 0; // b contains 0

b += 1; // b contains 1

b++; // b contains 2

console.log(b); // show 2
```

Type conversion

An expression's evaluation can result in **type conversion**. These are called implicit conversions, as they happen automatically without the programmer's intervention. For example, using the + operator between a string and a number causes the concatenation of two values into a result: a string.

```
var f = 100;

console.log("The variable f contains the value: " + f);
```



JavaScript is very tolerant in terms of type conversion. However, sometimes conversion isn't possible. If a number fails to convert, you'll get the result NaN (Not a Number).

```
var g = "five" * 2;

console.log(g); // Shows NaN
```

Sometimes you'll wish to convert the value of another type. This is called explicit conversion. JavaScript has the Number() and String() commands that convert to numbers and strings.

```
var h = "5";
```

```
console.log(h + 1); // Concatenation: shows the string "51"

h = Number("5");

console.log(h + 1); // Addition: show the number 6
```

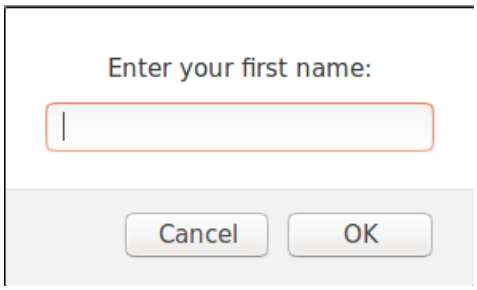
User interactions

Entering and displaying information

Once you start using variables, you can write programs that exchange information with the user.

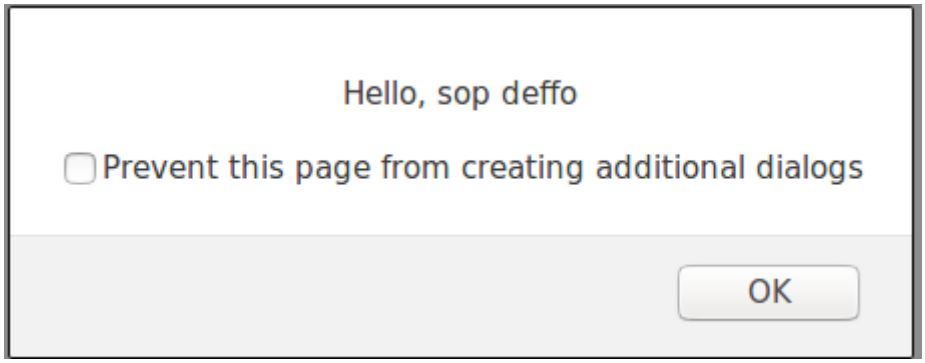
In the chapter-3/js directory, create a file named hello.js with the contents below.

```
var name = prompt ("Enter your first name:");

alert ("Hello, " + name);
```



This is the result of the JavaScript command prompt("Enter your first name:").

Type your name and click OK. You'll then get a personalized greeting.



The value you entered in the first dialog box has been stored as a string in the variable name. The JavaScript command alert() then triggered the display of the second box, containing the result of the concatenation of the string "Hello, " with the value of the name variable.

Naming variables

To close this part, let's discuss variable naming conventions. The computer doesn't care about variable names. You could name your variables using the classic example of a single letter (a, b, c...) or choose absurd names like burrito or puppies kittens90210.

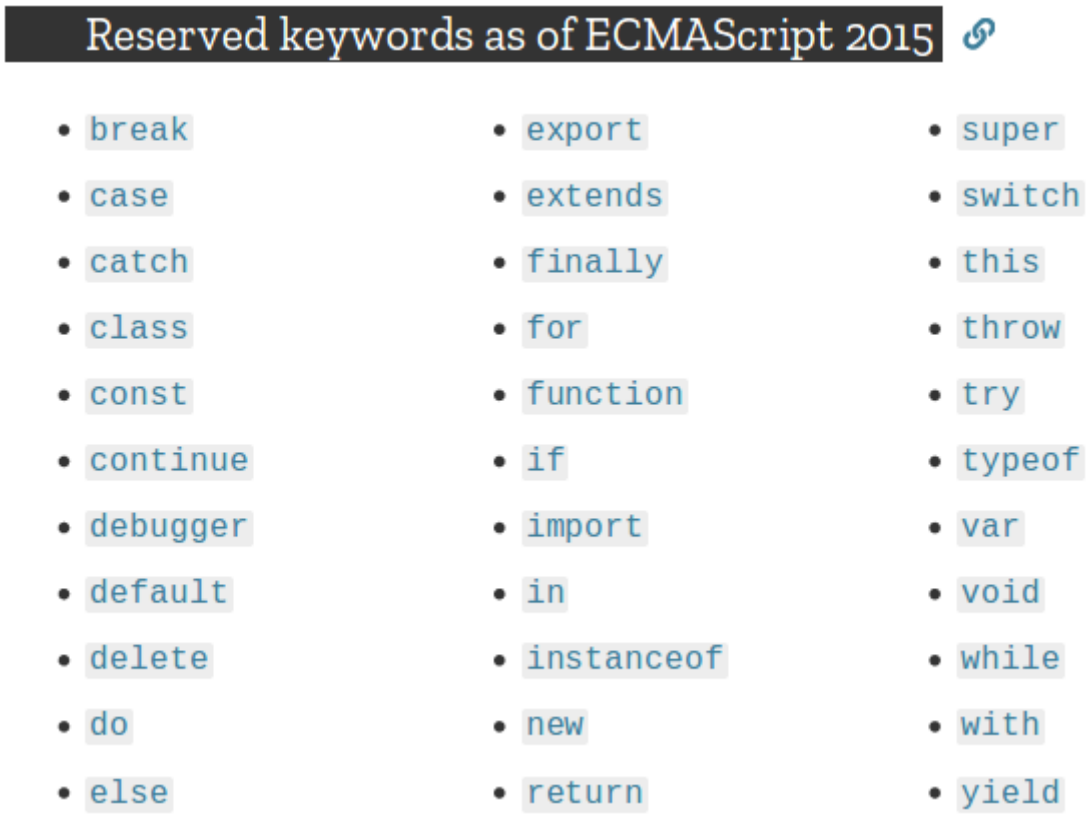
Nonetheless, naming variables clearly can make your code much easier to read. Check out these two examples:

```
var nb1 = 5.5;

var nb2 = 3.14;

var nb3 = 2 * nb2 * nb1;

console.log(nb3);
```

```
var radius = 5.5;

var pi = 3.14;

var circumference = 2 * pi * radius;

console.log(circumference);
```

They function in the same way, but the second version is much easier to understand. What are some good ways to name variables?

- **Choose meaningful names**
  The most important rule is to give each variable a name that reflects its role. This is the case in the second example above, where you use the word "radius" to actually indicate...well, a radius.
- **Don't use reserved words**
  The keywords of JavaScript are reserved names. They should not be used as variable names. Here's the list of reserved words in JavaScript.



- **Follow conventions**

  It can take several words to describe the roles of certain variables. The most common way to account for these variables is to use camelCase, based on two main principles:

  - All variable names begin with a lowercase letter.
  - If the name of a variable consists of several words, the first letter of each word (except the first word) is uppercase.

Like many languages, JavaScript is case sensitive. For example, myVariable and myvariable are two different variable names. Be careful!

2.4 Add conditions

Up until now, all the code in our programs has been executed chronologically. Let's enrich our code by adding conditional execution!

What's a condition?

Suppose we want to write a program that makes a user enter a number and then displays a message if the number is positive. Here is the corresponding algorithm.

Enter a number

If the number is positive

    Display Message

The message should display only if the number is positive: this means it's subject to a condition.

The *if* statement

Here's how you translate the program to JavaScript.

```
var number = Number (prompt("Enter a number:"));

if (number > 0) {

    console.log (number + " is positive");

}
```

Most basically, conditional syntax looks like this:

```
if (condition) {

    // Statements executed when the condition is true

}
```

The pair of opening and closing braces defines what is called a **block of code** associated with an if statement. This statement represents a test. It can result in the following: "If the condition is true, then execute the instructions contained in the code block."

The condition is always placed in parentheses after the if.

The statements within the associated code block are shifted to the right. This practice is called **indentation** and helps make your code more readable.

Conditions

A condition is an expression that evaluates whether something is true or false. When the value of a condition is true, we say that this condition is satisfied.

We have already studied numbers and strings, two types of data in JavaScript. **Booleans** are another type. This type has two possible values: **true** and **false**.

Any expression producing a Boolean value (either true or false) can be used as a condition in an if statement. If the value of this expression is true, the code block associated with it is executed.

Boolean expressions can be created using the comparison operators shown in the following table.

Operator	**Meaning**
===	Equal to
!==	Not equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater or equal to

It's easy to confuse comparison operators like === (or ==) with the assignment operator

(=). They're very, very different. Be warned!

In your chapter-3 directory, modify the following code to replace > with >= and change the message, then test it with the number 0.

```
var number = Number(prompt("Enter a number:"));

if (number >= 0) {
```

```
    console.log(number + " is positive or zero.");

}
```

The message appears in the console, which means that the condition (number >= 0) was true!

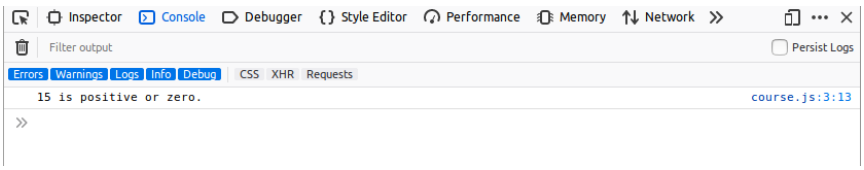
Alternative conditions

You'll often want to have your code execute one way when something's true and another way when something's false.

Else

Let's enrich our sample with different messages depending on whether the number is positive or not.

```
var number = Number(prompt("Enter a number:"));

if (number > 0) {

    console.log(number + " is positive");

}

else {

 console.log(number + " is negative or zero");

}
```



Test this code with a positive number, negative number, and zero, while watching the result in the console. The code executes differently depending if the statement (number > 0) is true or false.

This all happens by using the keyword else along with an initial if:

```
if (condition) {

   // code to run when condition is true

}

else {

   // code to run when condition is not true

}
```

You can translate an if / else statement like this: "If the condition is true, then execute this first set of code; otherwise, execute this next set of code."

Nesting conditions

You can go next-level and display a specific message if the entered number is zero. Use else if for a second conditional. See this example, which has a positive test case, negative test case, and a last resort of the number being zero:

```
var number = Number(prompt("Enter a number:"));

if (number > 0) {

   console.log(number + " is positive");

} else if (number < 0) {

   console.log(number + " is negative");

} else {

   console.log(number + " is zero");

}
```

Add additional logic

"And" operator

Suppose you want to check if a number is between 0 and 100. You're essentially checking if it's "greater than or equal to 0" *and* "less than or equal to 100."

Here's how you'd translate that same verification into JS.

```
if ((number >= 0) && (number <= 100)) {

   console.log(number + " is between 0 and 100, both included");

}
```

The && operator ("and") can apply to both types of boolean values. true will only be the result of the statement if both conditions are true.

```
console.log(true && true); // true

console.log(true && false); // false

console.log(false && true); // false

console.log(false && false); // false
```

"Or" operator

Now imagine you want to check that a number is outside the range of 0 and 100. To meet this requirement, the number should be less than 0 *or* greater than 100.

Here it is, translated into JavaScript:

```
if ((number < 0) || (number > 100)) {

   console.log(number + "is not in between 0 and 100");

}
```

The || operator ("or") makes statements true if at least one of the statements is true. For example:

```
console.log(true || true); // true
```

```
console.log(true || false); // true

console.log(false || true); // true

console.log(false || false); // false
```

"Not" operator

There's another operator for when you know what you *don't* want: the not operator! You'll use a !for this.

```
if (!(number > 100)) {

   console.log(number + " is less than or equal to 100");

}
```

Therefore:

```
console.log(!true); // false

console.log(!false); // true
```

Wild, right?!

Multiple choices

Let's write some code that helps people decide what to wear based on the weather using if/else.

```
var weather = prompt("What's the weather like?");

if (weather === "sunny") {

   console.log("T-shirt time!");

} else if (weather === "windy") {

   console.log("Windbreaker life.");

} else if (weather === "rainy") {
```

```
    console.log("Bring that umbrella!");
} else if (weather === "snowy") {
    console.log("Just stay inside!");
} else {
    console.log("Not a valid weather type");
}
```

Add this code in the course.js file, and test it out.

When a program should trigger a block from several operations depending on the value of an expression, you can write it using the JavaScript statement switch to do the same thing.

```
var weather = prompt("What's the weather like?");
switch (weather) {
case "sunny":
    console.log("T-shirt time!");
    break;
case "windy":
    console.log("Windbreaker life.");
    break;
case "rainy":
    console.log("Bring that umbrella!");
    break;
case "snowy":
```

```
    console.log("Just stay inside!");
    break;
default:
    console.log("Not a valid weather type");
}
```

If you test it out, the result will be the same as the previous version.

The word switch kicks off the execution of one code block among many. Only the code block that matches the relevant situation will be executed.

```
switch (expression) {
case value1:
    // code when the expression matches value1
    break;
case value2:
    // code when the expression matches value2
    break;
...
default:
    // code for when neither case matches
}
```

You can set as many cases as you want! The word default, which is put at the end of switch, is optional. It allows you to handle errors or unexpected values.

break; in each block is important so you get out of the switch statement!

2.5 Repeat statements

In this part, we'll look at how to execute code on a repeating basis!

Introduction

If you wanted to write code that displayed numbers between 1 and 5, you could do it with what you've already learned:

```
console.log(1);

console.log(2);

console.log(3);

console.log(4);

console.log(5);
```

This is pretty tiresome though and would be much more complex for lists of numbers between 1 and 1000, for example. How can you accomplish the same thing more simply?

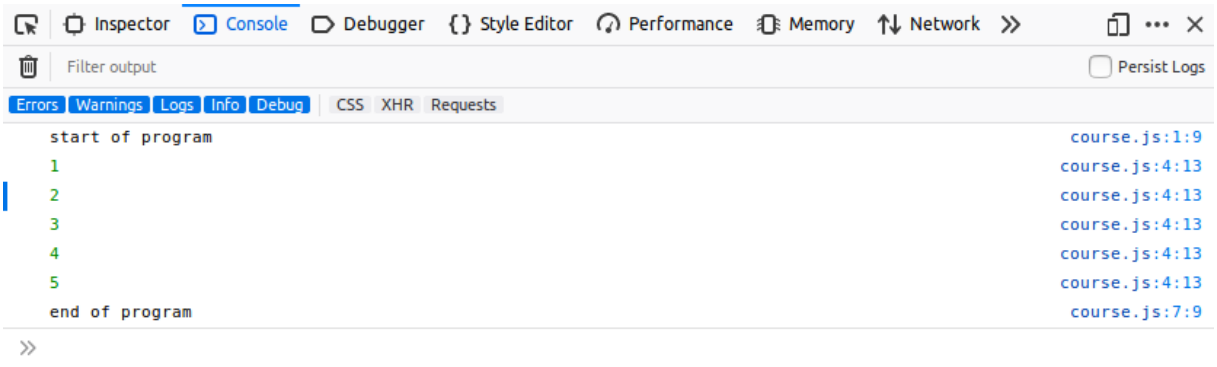
JavaScript lets you write code inside a **loop** that executes repeatedly until it's told to stop. Each time the code runs, it's called an **iteration.**

"While" loops

A while loop lets you repeat code while a certain condition is true. Here's a sample program written with a while loop.

```
console.log("start of program");

var number = 1;

while (number <= 5) {

   console.log(number);

   number++;
```

```
console.log("end of program");
```



In your chapter-3 folder, add the above file contents to test it out.

You'll use the following syntax to write a while loop.

```
while (condition) {

  // code to run while the condition is true

}
```

Before each loop iteration, the condition in parentheses is evaluated to determine whether it's true or not. The code associated with a loop is called its body!

- If the condition's true, the code in the while loop's body runs. Afterward, the condition is re-evaluated to see if it's still true or not. The cycle continues!
- If the condition is false, the code in the loop stops running or doesn't run.

The loop body must be placed within curly braces, except if it's only one statement. For now, *always* use curly braces for your loops.

"For" loops

You'll often need to write loops with conditions that are based on what happens in the loop body, like in our example. JavaScript offers another loop type to account for this: the for loop.

How it works

Here's the proper for loop syntax:

```
for (initialization; condition; final-expression) {
```

```
  // code to run when condition is true

}
```

This is a little more complicated than while loop syntax:

- Initialization only happens once, when the code first kicks off.
- The condition is evaluated once *before* the loop runs each time. If it's true, the code runs. If not, the code doesn't run.
- The final expression is evaluated *after* the loop runs each time. It's often used to update a counter variable, as we saw in the while loop example.

The variable used during initialization, condition, and the final expression is often called a counter.

The counter within a for loop is often called i.

```
for (var counter = 1; counter <= 5; counter++) {

  console.log(counter);

}
```

Common errors

Infinite loops

The main risk with while loops is a producing an **infinite loop,** meaning the condition is always true, and the code runs forever. This will crash your program! For example, let's say you forget a code line that increments the number variable. If you try to open the page in Chrome, the browser will freak out, and you'll strain your system.

```
var number = 1;

while (number <= 5) {

  console.log(number);

  // The number variable is always 1, so the loop will run each time...forever.
```

```
}
```

Manipulating a "for" loop counter

Imagine that you accidentally modify the loop counter in the loop body.

```
for (var counter = 1; counter <= 5; counter++) {

   console.log(counter);

   counter++; // the counter variable changes within the loop body

}
```

Each time the loop runs, the counter variable is incremented twice: once in the body and once in the expression after the loop runs. When you're using a for loop, you'll almost always want to omit anything to do with the counter from the body of your loop. Just leave it in that first line!

Which loop should I use?

For loops are great because they include the notion of counting by default, which avoids the problem of infinite loops. However, it means you have to know how many times you want the loop to run as soon as you write your code. For situations where you don't already know how many times the code should run, while loops make sense. As for a good while loop example, let's imagine a user will enter letters over and over until they enter X:

```
var letter = "";

while (letter !== "X") {

   letter = prompt("Type any letter or X to exit:");

   console.log(letter);

}
```

You can't know how many times it'll take for the user to enter X, so while is generally good for loops that depend on user interaction. Ultimately, choosing which loop to use depends on

context. All loops can be written with `while` , but if you know in advance how many times you want the loop to run, `for` is the best choice.

III. Create functions and objects

3.1 Write functions

In this part, you'll learn how to break down a program into subparts called functions.

Introduction: the role of functions

To understand why functions are important, check out this example

```
Begin
    Get out the rice cooker
    Fill it with rice
    Fill it with water
    Cook the rice
    Chop the vegetables
    Stir-fry the vegetables
    Taste-test the vegetables
        If the veggies are good
            Remove them from the stove
        If the veggies aren't good
            Add more pepper and spices
        If the veggies aren't cooked enough
            Keep stir-frying the veggies
```

Heat the tortilla

Add rice to tortilla

Add vegetables to tortilla

Roll tortilla

End

Here's the same general idea written more simply.

Begin

Cook rice

Stir-fry vegetables

Add fillings

Roll together

End

The first version details all the individual actions that make up the cooking process. The second breaks down the recipe into **broader steps** and introduces concepts that could be re-used for other dishes as well like *cook*, *stir-fry*, *add*, and *roll*.

Our code so far has mimicked the first example (in that it's been very literal), but it's time to start modularizing our example into sub-steps so we can re-use bits and pieces as needed. In JavaScript, these sub-steps are called **functions**!

Functions

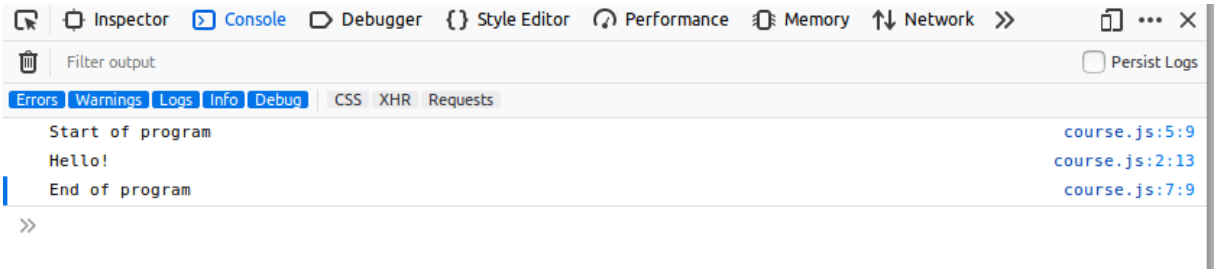
A **function** is a group of instructions that performs a particular task.

Here's a basic example of a function.

```
function sayHello() {

    console.log("Hello!");
```

```
}

console.log("Start of program");

sayHello();

console.log("End of program");
```

Go ahead and add this code to your chapter-3 JavaScript file. Opening the corresponding HTML file in your browser should display this:



Declaring a function

Check out the first lines of the example above.

```
function sayHello() {

  console.log("Hello!");

}
```

This creates a function called sayHello(). It consists of only one statement that will make a message appear in the console: "Hello!"

Creating a function is called **declaration**:

```
// Declaring a function called myFunction

function myFunction() {

  // Function actions

}
```

The declaration of a function is performed using the JavaScript keyword function, followed by the function name and a pair of parentheses. Instructions that make up the function constitute the **body** of the function. These instructions are enclosed in curly braces and indented.

Calling a function

If a function is defined in a forest, but there's no one to call it, is it real?! **Functions must be called in order to actually run.**

```
console.log("Start of program");

sayHello();

console.log("End of program");
```

The first and third statements explicitly display messages in the console. The second line makes a call to the function sayHello() .

You can **call** a function by writing the name of the function followed by a pair of parentheses.

```
// ...

myFunction(); // Call myFunction

// ...
```

Calling a function triggers the execution of actions listed therein.

Just remember for each function:

1. Declare it.
2. Call it.

Function contents

Return value

Here is a variation of our example program.

```
function sayHello() {
```

```
    return "Hello!";
}

console.log("Start of program");

var result = sayHello();

console.log(result);

console.log("End of program");
```

Run this code, and you'll see the same result as before.

In this example, the body of the sayHello() function has changed: The statementconsole.log("Hello!") was replaced by return "Hello!".

The keyword return indicates that a **return value** will be given, which is specified immediately after the word.

```
// Declare myFunction

function myFunction() {

    // Calculate return value

    // ...

    return returnValue;

}

// Get return value from myFunction

var value = myFunction();

// ...
```

This return value can be any type (number, string, etc.). However, a function can return only one value.

If you try to retrieve the return value of a function that does not actually have one, you get the JavaScript value undefined. This isn't necessarily an error; it just means the function may perform certain operations without actually outputting anything specific at the end.

```
function myFunction() {

   // No value returned in this function

}

var result = myFunction();

console.log(result); // Will be undefined
```

A function that returns no value is sometimes called a **procedure**.

A function stops running immediately after a return statement.

Local variables

You can declare variables inside a function, as in the example below.

```
function sayHello() {

   var message = "Hello!";

   return message;

}

console.log(sayHello());
```

The function sayHello() declares a variable named message and returns its value.

The variables declared in the body of a function are called **local variables**. They can be used only within the function itself!

```
function sayHello() {
```

```
    var message = "Hello!";

    return message;

}



console.log(sayHello());

console.log(message); // Error: the message variable doesn't exist here
```

Each function call will declare a local variable, each of them separate.

The **scope of a variable** includes all the places where the variable is accessible. The scope of a local variable is limited to the body of the function inside of which it is declared. If you try to use it outside the function, you won't be able to!

Not being able to use local variables outside the functions in which they are declared may seem like a limitation. Actually, it's a good thing! This means functions can be designed as autonomous and reusable.

Passing parameters

A **parameter** is information that the function needs in order to work. The function parameters are defined in parentheses immediately following the name of the function. You can then use the parameter value in the body of the function!

You supply the parameter value when calling the function, at which point we say that we're "**passing** an argument."

Parameter: the thing passed within the function **definition**

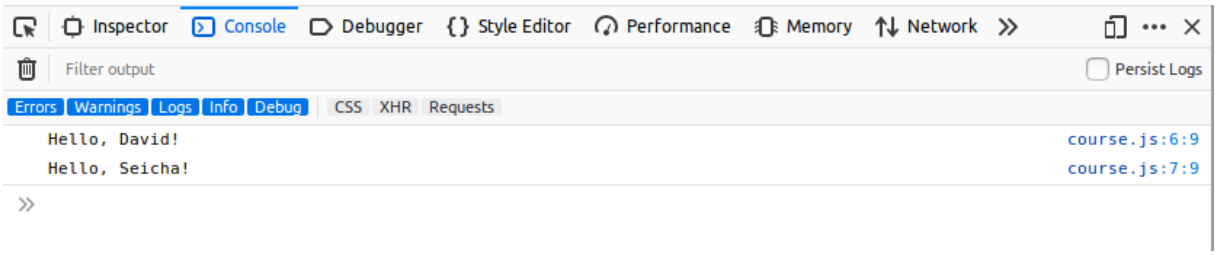
Argument: the thing passed when the function is **called**

Let's edit the above example to add a personalized greeting:

```
function sayHello(name) {

    var message = "Hello, " + name + "!";

    return message;
```

```
}

console.log(sayHello("David"));

console.log(sayHello("Seicha"));
```

The function declaration of sayHello() now contains a parameter called name. Test it out and you'll get the following result:



You can also pass multiple parameters to a function!

```
function sayHello(firstName, lastName) {

   var message = "Hello, " + firstName + lastName + "!";

   return message;

}

console.log(sayHello("David", "Goldman"));

console.log(sayHello("Seicha", "Miller"));
```

Be sure that you pass multiple arguments in the same order as they're defined in the function!

3.2 work with strings

A lot of code you'll write involves modifying strings, or chains of characters. Let's look at how!

String recap

Let's recap what we already know about strings:

- A *string* represents text.

- In JavaScript, a string is defined by placing text within single quotes ('I am a string') or double quotes ("I am a string").
- You can use special characters within a string by prefacing them with \ (backslash) followed by another character. For example, use \ n to add a linebreak.

- Use + to concatenate (combine) two strings together.

Strings are much more versatile beyond these basic uses.

String length

To find the length of a string (the number of characters in it), call .length on it. The length will be returned as an integer.

```
console.log("ABC".length); // will be 3

console.log("I am a string".length); // will be 13
```

Add the above contents to your course.js file in chapter-6/js . Open up the HTML equivalent in your browser to test it out!

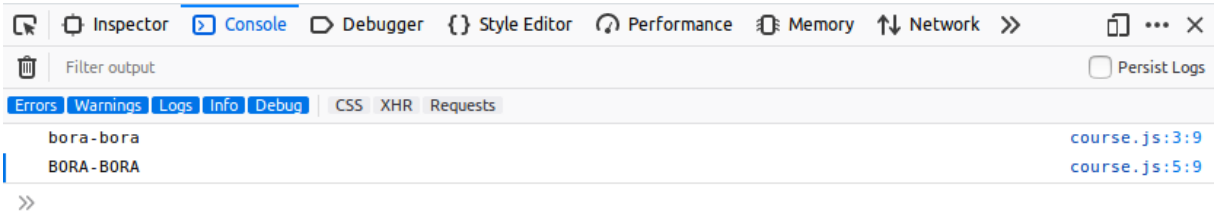
.length can also be applied to string variables, *and* the results can be stored in variables for later use.

Using a period (.) to call a method or property is called dot notation!

Converting string case

You can convert a string's text to lowercase by calling .toLowerCase() on it. Alternatively, you can do the same thing with .toUpperCase() to convert it to uppercase. Both will return a new string.

```
var originalWord = "Bora-Bora";

var lowercaseWord = originalWord.toLowerCase();

console.log(lowercaseWord); // will be "bora-bora"

var uppercaseWord = originalWord.toUpperCase();
```



console.log(uppercaseWord); // will be "BORA-BORA"

Notice you also used dot notation to convert the string to upper or lowercase. However, you used parentheses after toLowerCase() or toUpperCase(). That's because they're **methods**, not properties!

Compare two strings

You can compare two strings with the === operator. This returns a boolean value: true if the strings are equal, false if they are not.

var word = "koala";

console.log(word === "koala"); // will be true

console.log(word === "kangaroo"); // will be false

Beware: string comparison is **case sensitive!** You'll have to pay attention to your lower and uppercase letters.

Identify a particular character

You can think of a string as a set of characters. Each character is identified by a number called an **index.**

Here's the trick:

The first character in a string will be index number **0** -- not 1.

Therefore, the following index numbers would apply to the string "dogs":

- 0: d
- 1: o
- 2: g

- 3: s

The index of the last character in a string would be the string's length minus 1.

Browse a character string by character

You know how to identify a character by its index. What if you want to access all characters one-by-one? You could individually access each letter, as seen above:

```
var name = "Sarah"; // 5 characters

console.log(name[0]);

console.log(name[1]);

console.log(name[2]);

console.log(name[3]);

console.log(name[4]);
```

This is impractical if your string contains more than a few characters. What's a better solution for repeated access to characters?

Does the word "repeat" bring to mind a former concept? **Loops!**

You can write a loop to access each character of a string. for or while ? The choice depends largely on the number of times the loop needs to run. If you know in advance, a for loop is a good choice. We know here that the loop will need to run for each character in the string...so for its length!

```
for (var i = 0; i < myString.length; i++) {

}
```

The loop counter i ranges from 0 (the index of the string's first character) to string length - 1 (index of the last character). When the counter value equals the string length, the expressioni < myString.length becomes false, and the loop ends.

3.3 Notion of Object

Have you heard about object oriented programming, inheritance, or classes? Whether the answer's yes or no, learn more about them in this chapter!

Introduction

What's an object?

Think about objects in the non-programming sense, like a pen. A pen can have different ink colors, be manufactured by different people, have a different tip, and many other properties.

Similarly, an object in programming is **an entity that has properties**. Each property defines a characteristic of the object. A property can be *information* associated with the object (the color of the pen) or *action* (the pen's ability to write).

What does this have to do with code?

Object-oriented programming (OOP for short) is **a way to write programs using objects**. When using OOP, you write, create, and modify objects, and the objects make up your program.

OOP changes the way a program is written and organized. So far, you've been writing function-based code, sometimes called the procedural programming. Now let's discover how to write object-oriented code.

JavaScript and objects

Like many other languages, JavaScript involves programming objects, so we can say it's an object-oriented language. It provides a number of predefined objects while also letting you create your own.

Creating an object

Here is the JavaScript representation of a blue Bic ballpoint pen.

```
var pen = {

  type: "ballpoint",

  color: "blue",
```

```
    brand: "Bic"
};
```

Create a new object in JavaScript by setting its properties within a pair of curly braces: {...};

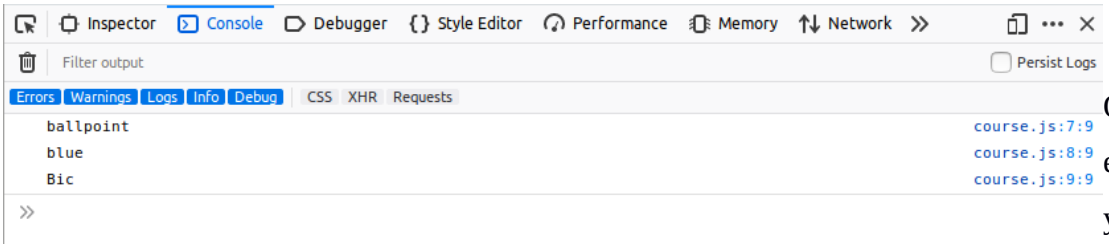
The above code defines a variable named pen whose value is an object: you can therefore say pen is an object. This object has three properties: type, color , and brand. Each property has a name and a value and is separated by a comma , (except the last one).

Getting a value

After creating the object, you can access the value of its properties using dot notation such as myObject.myPro

perty .

```
console.log(pen.type); // will be "ballpoint"
console.log(pen.color); // will be "blue"
console.log(pen.brand); // will be "Bic"
```



Once you know how to access properties, you can start combining them with other elements, like strings as part of a sentence.

```
var pen = {
    type: "ballpoint",
    color: "blue",
    brand: "Bic"
```

```
};

console.log("My pen is a " + pen.color + " " + pen.brand + " " + pen.type + " pen.");
```

Modifying objects

Once an object is created, you can change the values of its properties with the syntaxmyObject.myProperty = newValue.

```
var pen = {
    type: "ballpoint",
    color: "blue",
    brand: "Bic"
};

console.log("The pen's a " + pen.color + " " + pen.brand + " " + pen.type + " pen.");

pen.color = "red"; // modify the pen color property

console.log("The pen's a " + pen.color + " " + pen.brand + " " + pen.type + " pen.");
```

JavaScript even offers the ability to dynamically add new properties to an already created object.

```
var pen = {
    type: "ballpoint",
    color: "blue",
    brand: "Bic"
};

console.log("My pen is a " + pen.color + " " + pen.brand + " " + pen.type + " pen.");
```

```
pen.price = "2.5"; // set the pen price property

console.log("My pen costs " + pen.price);
```

Example: cake

Pens are fine, but cake is cooler. Let's create a cake in JavaScript that has several properties:

- flavor, like vanilla, chocolate, etc.
- price in dollars
- layers (1, 2, 3...10?!)

Creating a cake

Here's my first cake created in JavaScript:

```
var cake = {
   flavor: "strawberry",
   layers: 2,
   price: "$10",
   occasion: "birthday",
}
```

You can assign numbers, strings, and even other objects to properties!

However, I'm working with a customer who wants a much more aggressive birthday cake:

```
var cake = {
   flavor: "strawberry",
   levels: 2,
   price: "$10",
   occasion: "birthday",
```

```
};

console.log("The " + cake.occasion + " cake has a " + cake.flavor + " flavor, " + cake.levels + " layers, and costs " + cake.price + ".");

// The cake is actually for a wedding!

cake.occasion = "wedding";

// Its number of layers and price both go up.

cake.levels = cake.levels + 8;

cake.price = "$50";

console.log("The " + cake.occasion + " cake has a " + cake.flavor + " flavor, " + cake.levels + " layers, and costs " + cake.price + ".");
```

Add this contents to your JavaScript file and open it in your browser to see the result in the JavaScript console.

Methods on objects

In the above code, we had to write lengthy console.log statements each time to show the cake description. There's a cleaner way to accomplish this.

Adding a method to an object

Observe the following example.

```
// Describe a cake

function describe(cake) {

   var description = "The " + cake.occasion + " cake has a " + cake.flavor + " flavor, " + cake.levels + " layers, and costs " + cake.price + ".";

   return description;

}
```

```
console.log(describe(cake));
```

The function describe() takes an object as a parameter. We pass it the cake, and it accesses that object's properties and prints them out in that sentence.

Now for an alternative approach: creating a describe property that houses a method.

```
var cake = {
    flavor: "strawberry",
    levels: 2,
    price: "$10",
    occasion: "birthday",
    // Describe the cake
    describe: function () {
        var description = "The " + this.occasion + " cake has a " + this.flavor + " flavor, " + this.levels + " layers, and costs " + this.price + ".";
        return description;
    }
};
console.log(cake.describe());
// The cake is actually for a wedding!
cake.occasion = "wedding";
// Its number of layers and price both go up.
cake.levels = cake.levels + 8;
```

```
cake.price = "$50";

console.log(cake.describe());
```

Now our object has a new property available to it, describe. The value of this property is a function that returns a text description of the cake.

A property whose value is a function is called a **method**.

Remember the parentheses, even if empty, when calling a method!

The keyword *this*

Now look at the body of the describe() method on our cake object. You see a new word: this. This is automatically set by JavaScript inside a method and represents **the object on which the method was called**.

3.4 Store data in array

This part will introduce you to the tables, or **arrays,** used in many computer programs to store data.

Introduction to arrays

Imagine you want to create a list of all the movies you've seen this year.

One solution would be to create film variables:

```
var film1 = "Jurassic Park";

var film2 = "Titanic";

var film3 = "Toy Story";

// ...
```

If you're a movie buff, you may quickly find yourself with too many variables in your program. The worst part is that these variables are completely independent from one another.

Another possibility is to group the films in an object.

```
var films = {
   film1: "Jurassic Park",
   film2: "Titanic,
   film3: "Toy Story",
   // ...
};
```

This time, the data is centralized in the object films. However, the names of its properties (film1,film2,film3...) are unnecessary and repetitive.

We must find a solution to store items together without naming them individually!

Luckily, there is one: use an array. An **array** is a data type that can store a set of elements.

Manipulating arrays in JavaScript

In JavaScript, an array is an object that has special properties.

Create arrays

How to create our list of films in the form of a table.

```
var films = ["Jurassic Park", "Titanic", "Toy Story"];
```

Create a table with a pair of square brackets: []. Everything within the brackets makes up the array.

You can store different types of elements within an array, including strings, numbers, and booleans:

```
var elements = ["Hello", 7, true];
```

Since an array contains multiple things, it's good to name the array using a plural (for example, films).

Identify an array's size

The number of elements stored in a table is called its size. Here's how to access it.

```
var films = ["Jurassic Park", "Titanic", "Toy Story"];

console.log(films.length); // will be 3
```

Add the above code to your chapter-9JavaScript file if you have one and open the HTML file in your browser to show the result.

You access the size of an array via its lengthproperty.

Remember, we also used this length property to get the size of a string!

Of course, this length property returns 0 for an empty array.

```
var emptyArray = []; // create an empty array

console.log(emptyArray).length); // will be 0
```

Access an element in an array

Each item in an array is identified by a number called its **index**. We can represent an array as a set of boxes, each storing a specific value and associated with an index. Here is how one might represent the films array:

Index	0	1	2
Value	Jurassic Park	Titanic	Toy Story

You can access a particular element by passing its index to the array name within square brackets:

```
var films = ["Jurassic Park", "Titanic", "Toy Story"];

console.log(films[0]); // will be "Jurassic Park"

console.log(films[1]); // will be "Titanic"

console.log(films[2]); // will be "Toy Story"
```

That's exactly the way you accessed characters in a string! The same golden rules apply:

- The index of the first element of an array is 0, not 1.
- The highest index number is the array's length minus 1.
- Using an invalid index to access a JavaScript array element returns the value undefined.

```
// ...

console.log(films[3]); // will be undefined
```

Browse an array

There are two ways to browse an array element by element.

The first is to use a for loop as discussed previously in the course.

```
var films = ["Jurassic Park", "Titanic", "Toy Story"];

for (var i = 0; i < films.length; i++) {

   console.log(films[i]);

}
```

The for loop runs through each element in the array starting with index 0 all the way up to the length of the array minus 1, which will be its last value.

Add an element to an array

Let's say you just watched *The Revenant* and you want to add it to the list. Here's how you'd do so:

```
var films = ["Jurassic Park", "Titanic", "Toy Story"];

films.push("The Revenant");

console.log(films[3]); // will show "The Revenant"
```

You add a new item to an array with the method push(). You pass the new element to be added as a parameter to the method.

Object arrays

An array can store any type of item, including objects and even other arrays!

Say you want to also store the year of release of each film seen this year. One solution is to store these dates directly in the array as their own items. **var films = ["Jurassic Park", 1993, "Titanic", 1997, "Toy Story", 1995];**

However, this makes things tricky since now, when accessing the element at a particular index, you can't be sure which year corresponds to which film. Imagine now adding directors and other elements. Too tough!

A better approach would be to represent each film as an object.

```
var Film = {

   // Initialise le film

   init: function (title, year) {

      this.title = title;

      this.year = year;

   },

   // Renvoie la description du film

   describe: function () {

      var description = this.title + " (" + this.year + ")";

      return description;

   }

};

var film1 = Object.create(Film);

film1.init("Jurassic Park", 1993);
```

```
var film2 = Object.create(Film);

film2.init("Titanic", 1997);

var film3 = Object.create(Film);

film3.init("Toy Story", 1995);
```

The Film object is a model for each item. init() gives each film a title and a year, and the describe() method writes out each film as "*Title* (year)".

Objects film1,film2 and film3 are created with Film as a prototype, and they therefore have its properties.

Now you can create an array called films containing our objects and then use it to display the description of each film.

```
// ...

var films = [];

films.push(film1);

films.push(film2);

films.push(film3);

films.forEach(function (film) {

    console.log(film.describe());

});
```

In our example, the function associated with the method forEach() displays the result of the methoddescribe() on each object in the array of films.